Mail-Order Database

The mail-order database consists of the relations defined in the six schemes shown in Figure 1.3.

Figure 1.3 Mail-order database schemes.

```
EMPLOYEES(ENO,ENAME,ZIP,HDATE)
PARTS(PNO,PNAME,QOH,PRICE,LEVEL)
CUSTOMERS(CNO,CNAME,STREET,ZIP,PHONE)
ORDERS(ONO,CNO,ENO,RECEIVED,SHIPPED)
ODETAILS(ONO,PNO,QTY)
ZIPCODES(ZIP,CITY)
```

- The EMPLOYEES relation contains information about the employees of the company. The ENO attribute is the primary key. The ZIP attribute is a foreign key referring to the ZIPCODES table.
- The PARTS relation keeps a record of the inventory of the company. The record for each part includes its number and name as well as the quantity on hand, the unit price, and the reorder level. PNO is the primary key for this relation.
- The CUSTOMERS relation contains information about the customers of the mail-order company. Each customer is assigned a customer number, CNO, which serves as the primary key. The ZIP attribute is a foreign key referring to the ZIPCODES relation.
- The ORDERS relation contains information about the orders placed by customers, the employees who took the orders, and the dates the orders were received and shipped. ONO is the primary key. The CNO attribute is a foreign key referring to the CUSTOMERS relation, and the ENO attribute is a foreign key referring to the EMPLOYEES table.
- The ODETAILS relation contains information about the various parts ordered by the customer within a particular order. The combination of the ONO and PNO attributes forms the primary key. The ONO attribute is a foreign key referring to the ORDERS relation, and the PNO attribute is a foreign key referring to the PARTS relation.
- The ZIPCODES relation maintains information about the zip codes for various cities. ZIP is the primary key.

A sample from a mail-order database is shown in Figure 1.4.

Figure 1.4 A sample from a mail-order database.

Employees

ENO	ENAME	ZIP	HDATE
1000	Jones	67226	12-DEC-95
1001	Smith	60606	01-JAN-92
1002	Brown	50302	01-SEP-94

Parts

PNO	PNAME	QOH	PRICE	LEVEL
10506	Land Before Time I	200	19.99	20
10507	Land Before Time II	156	19.99	20
10508	Land Before Time III	190	19.99	20
10509	Land Before Time IV	60	19.99	20
10601	Sleeping Beauty	300	24.99	20
10701	When Harry Met Sally	120	19.99	30
10800	Dirty Harry	140	14.99	30
10900	Dr. Zhivago	100	24.99	30

Customers

CNO	CNAME	STREET	ZIP	PHONE
1111	Charles	123 Main St.	67226	316-636-5555
2222	Bertram	237 Ash Ave.	67226	316-689-5555
3333	Barbara	111 Inwood St.	60606	316-111-1234

Orders

ONO	CNO	ENO	RECEIVED	SHIPPED
1020	1111	1000	10-DEC-94	12-DEC-94
1021	1111	1000	12-JAN-95	15-JAN-95
1022	2222	1001	13-FEB-95	20-FEB-95
1023	3333	1000	20-JUN-97	null

Odetails

ONO	PNO	QTY
1020	10506	1
1020	10507	1
1020	10508	2
1020	10509	3
1021	10601	4
1022	10601	1
1022	10701	1
1023	10800	1
1023	10900	1

Zip codes

ZIP	CITY
67226	Wichita
60606	Fort Dodge
50302	Kansas City
54444	Columbia
66002	Liberal
61111	Fort Hays

Figure 2.8 Mail-order insert statements.

```
insert into  zipcodes values (67226,'Wichita');

insert into employees values
  (1000,'Jones',67226,'12-DEC-95');

insert into parts values
  (10506,'Land Before Time I',200,19.99,20);
insert into parts values
  (10507,'Land Before Time II',156,19.99,20);
insert into parts values
  (10508,'Land Before Time III',190,19.99,20);
insert into parts values
  (10509,'Land Before Time IV',60,19.99,20);

insert into customers values
  (1111,'Charles','123 Main St.',67226,'316-636-5555');

insert into orders values
  (1020,1111,1000,'10-DEC-94','12-DEC-94');

insert into odetails values (1020,10506,1);
insert into odetails values (1020,10507,1);
insert into odetails values (1020,10508,2);
insert into odetails values (1020,10509,3);
```

by an alias, separated by a space. These aliases can serve the purpose of the renaming operator in relational algebra. The `where` clause consists of the keyword `where` followed by a *search condition*, a Boolean expression involving constants, column names, and appropriate operators. The nature of the search condition will also be clarified by the examples in this section.

The informal semantics (the result set of tuples) of the simple `select` statement is computed as follows: The Cartesian product of the tables mentioned in the `from` clause is taken first. Then the selection criteria mentioned in the `where` clause are applied to the product, which results in a subset of the product. Finally, the columns

and expressions mentioned in the `select` clause are projected from this subset to form the result of the query.

Here are some examples of simple queries and their corresponding SQL `select` statements.

Query 2.1: Get pno and pname values of parts that are priced less than $20.00.

```
select pno,pname
from   parts
where  price < 20.00;
```

This is a simple example involving one table with a simple selection condition based on the column `price` and a projection of two columns, pno and pname.

Query 2.2: Get all the rows of the `employees` table.

```
select *
from   employees;
```

If all the columns of all the tables mentioned in the `from` clause are to be projected, the * symbol can be used in the `select` clause as shown.

Query 2.3: Get pno values of parts for which orders have been placed.

```
select distinct pno
from   odetails;
```

The `distinct` keyword can be used to eliminate duplicate answers, if present. By default, SQL does not eliminate duplicates. In many situations, it should not matter whether duplicates are removed; however, there are situations—for example, when using aggregate operations—where the result of the query depends on whether or not duplicates are removed.

Query 2.4: Get all the details of customers whose names begin with the letter "A".

```
select *
from   customers
where  cname like 'A%';
```

The `like` operator performs pattern matching in string data. A percent sign (%) indicates a match with zero or more spaces or characters, and the underscore sign (_) indicates a match with exactly one space or character. Additional examples of the usage of the `like` operator are

```
cname like '_ee%'  -- true if the second and
                      third letters of cname
                      are both 'e'.
cname like '%a%a%' -- true if cname contains
                      at least two 'a's.
```

Query 2.5: Get ono and cname values of customers whose orders have not yet been shipped (i.e., the shipped column has a null value).

```
select ono,cname
from   orders,customers
where  customers.cno = orders.cno and
       shipped is null;
```

The is null predicate tests for null values in a particular column. The is not null predicate can be used to check to see if a column has a non-null value. Using the comparison operators (=, !=, etc.) with the null value can give unpredictable results; hence, it is always advisable to use the is null and is not null predicates to check for null values.

Query 2.6: Get sid values of students who have scores between 50 and 70 points in any component of any course in which they have enrolled.

```
select sid
from   scores
where  points between 50 and 70;
```

The between predicate used in this example is a short form for the expression points >= 50 and points <= 70. There is a corresponding not between predicate that is the logical opposite of between.

Query 2.7: Get cname and ename pairs such that the customer with name cname has placed an order through the employee with name ename.

```
select distinct cname,ename
from   customers,orders,employees
where  customers.cno = orders.cno and
       employees.eno = orders.eno;
```

This is a query involving more than one table. The where clause consists of two conditions, referred to as *join conditions*, that relate the data present in the three tables being joined.

Query 2.8: For each odetail row, get ono, pno, pname, qty, and price values along with the total price for this item. The total price is simply the product of unit price and quantity.

```
select x.ono, x.pno, p.pname, x.qty,
       p.price, (x.qty * p.price) total
from   odetails x, parts p
where  x.pno = p.pno;
```

This query illustrates aliases used in the from clause and arithmetic expressions and naming a new column in the select clause. If the last expression in the select clause did not have the column name total, by default the column would be labeled by the text of the expression. Note that the aliases in the from clause in this example are not necessary. Since the only common column between the parts and odetails tables is pno, any references to it in the query must be preceded by the table name or the alias, such as x.pno. The prefixes on all other references are not necessary, even though they are shown in the example.

Query 2.9: Get all pairs of cno values for customers based in the same zip code.

```
select c1.cno, c2.cno
from   customers c1, customers c2
where  c1.zip = c2.zip and c1.cno < c2.cno;
```

The aliases used in this query are necessary because there is a reference to two different rows of the same table. The c1.cno < c2.cno condition is used to eliminate any redundant answers.

Query 2.10: Get pno values for parts that have been ordered by at least two different customers.

```
select distinct y1.pno
from   orders x1, orders x2, odetails y1, odetails y2
where  y1.pno = y2.pno and y1.ono = x1.ono and
       y2.ono = x2.ono and x1.cno < x2.cno;
```

Notice the extensive use of aliases in this example. The x1.cno < x2.cno condition is used to eliminate any redundant answers.

2.4.2 Sub-Selects

The search criteria in the where clause may itself contain a select statement. Such a select statement within the search criteria is referred to as a *sub-select*, and the